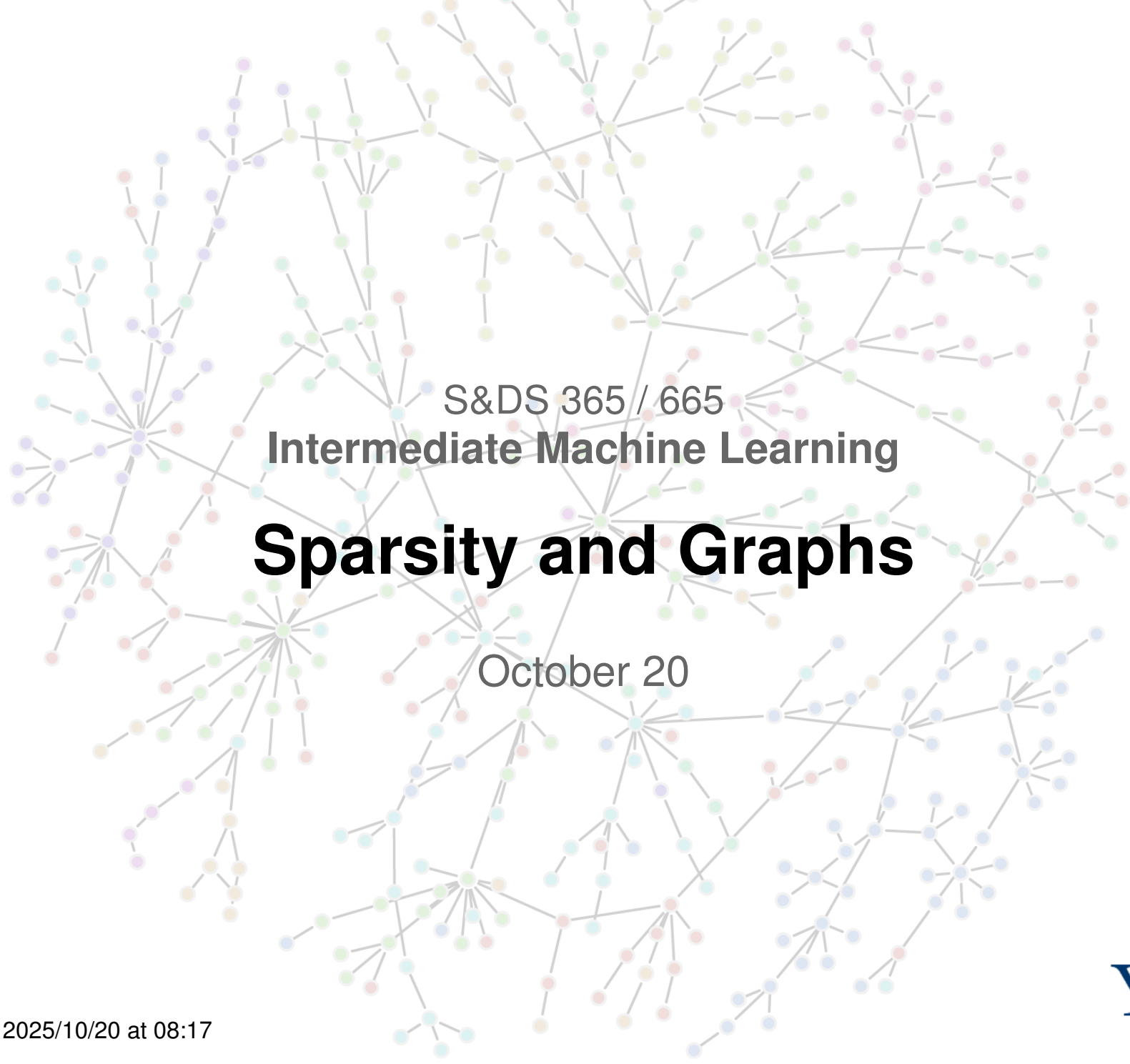


S&DS 365 / 665

Intermediate Machine Learning

Sparsity and Graphs

October 20

Yale

Version of 2025/10/20 at 08:17

Welcome back!

- Midterm exam grades posted tomorrow
- Mid-semester grades announced before Thurs
- Contact me @sds365 with any concerns about grades
- Assignment 3 out (VAEs); due next Thurs, Oct. 30
- Assignment 4 posted at same time

Outline

For today:

- Where have we been? Where are we going?
- Graphs of data/distributions

Where have we been?

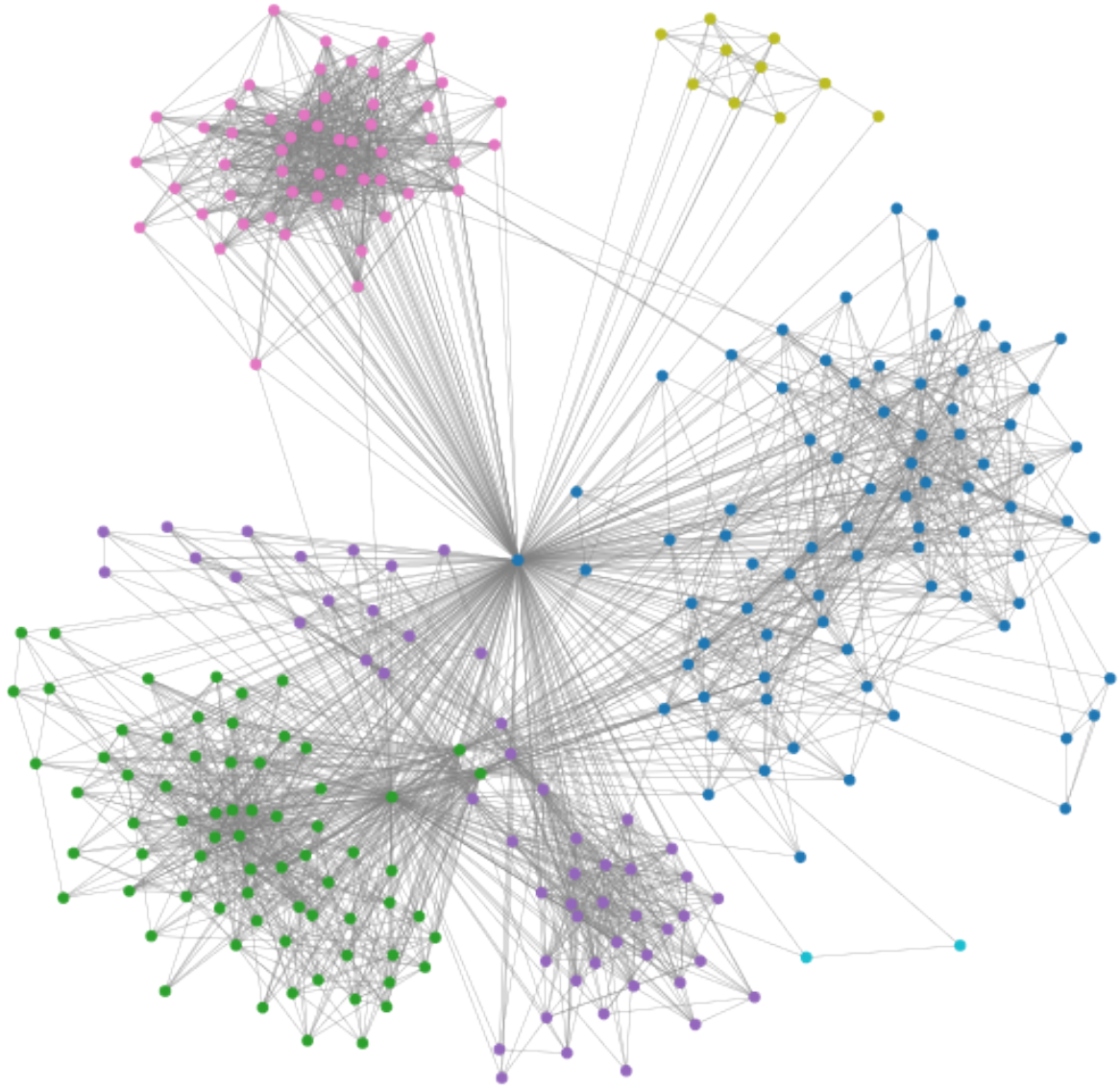
1	Aug 27, Aug 29	Sparse regression	Python elements Pandas and regression Lasso example	Wed: Course overview Fri: Sparse regression	Google Colab Basics PML Section 11.4 Notes on linear regression	
2	Sep 3	Smoothing and kernels	Smoothing example Using different kernels	Wed: Lasso (continued) and smoothing	PML Sections 16.3, 17.1 Notes on computing the lasso	Quiz 1
3	Sep 8, 10	Density estimation and Mercer kernels	Density estimation demo Mercer kernels (1/3) Mercer kernels (2/3) Mercer kernels (3/3)	Mon: Smoothing and density estimation Wed: Mercer kernels	Risk bounds for local smoothing Notes on Mercer kernels	Assn 1 (updated) Updated Prob 1.1
4	Sep 15, 17	Neural networks and overparameterized models	np-complete example (1/2) np-complete example (2/2) TensorFlow playground	Mon: Neural networks Wed: Double descent	PML Sections 13.1, 13.2 Notes on backpropagation Notes on double descent	Quiz 2
5	Sep 22, 24	Convolutional neural networks	Convolution demo CNN demo (1/2) CNN demo (2/2)	Mon: Convolutional neural networks Wed: CNNs and Gaussian Processes	PML Section 17.2 Notes on Bayesian inference Notes on nonparametric Bayes	Assn 1 in Assn 2 out ipynb converter
6	Sept 29, Oct 1	Gaussian processes and approximate inference	Parametric Bayes Gaussian processes Gibbs sampling for image denoising	Mon: Gaussian processes Wed: Recap of GPs Introduction to approximate inference	Notes on simulation	Quiz 3
7	Oct 6, 8	Variational inference	Variational autoencoders	Mon: Variational inference Wed: VAEs	PML Section 20.3 Notes on variational inference	Assn 2 in Assn 3 out ipynb converter
8	Oct 13	Midterm			Practice midterms	Oct 13: Midterm exam

Where are we going?

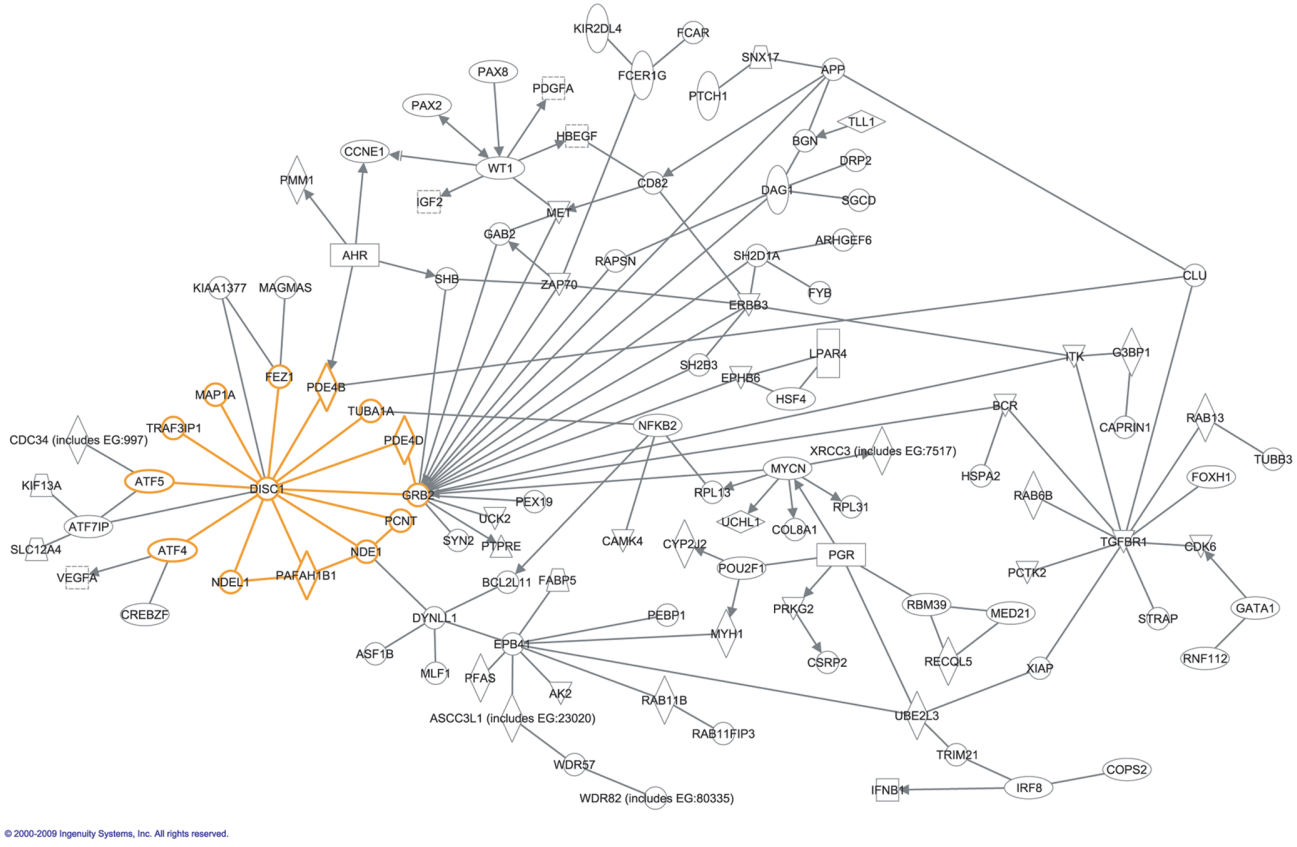
9	Oct 20, 22	Graphs and structure learning	Graphical lasso demo	Mon: Sparsity and graphs Wed: Discrete data and graph neural nets	Notes on graphs and structure learning Graph neural networks PML Section 23.4	
10	Oct 27, Oct 29	Deep reinforcement learning	Q-learning demo DQN demo	Mon: Reinforcement learning Wed: Deep reinforcement learning	Sutton and Barto, Section 6.5	Assn 3 in Assn 4 out ipynb converter
11	Nov 3, 5	Policy methods	Policy gradients demo Actor-critic demo	Mon: Policy gradient methods Wed: Actor-critic methods	Sutton and Barto, Section 13.1-13.3, 13.5	Quiz 4
12	Nov 10, 12	Sequential models	vanilla RNN Fakespeare GRU	Mon: HMMs and RNNs Wed: RNNs, GRUs, LSTMs, and all that	TensorFlow: Text generation Notes on HMMs and Kalman filters PML Chapter 15	
13	Nov 17, 19	Sequence-to-sequence models and Transformers	GPT-4 Python API	Mon: Sequence models and attention Wed: Transformers, LLM scaling, finetuning	Notes on mixtures PML Sections 15.4, 15.5	Assn 4 in Assn 5 out ipynb converter Quiz 5
14	Nov 24, 26	No class, Thanksgiving break				
15	Dec 1, 3	AI Societal issues	Transformer demo Minimal LLM decoder	Mon: LLM alignment Wed: Course wrap up		Assn 5 in
17	Dec 12, 9am	Final exam			Practice exams	Registrar: final exam schedule

Graphs

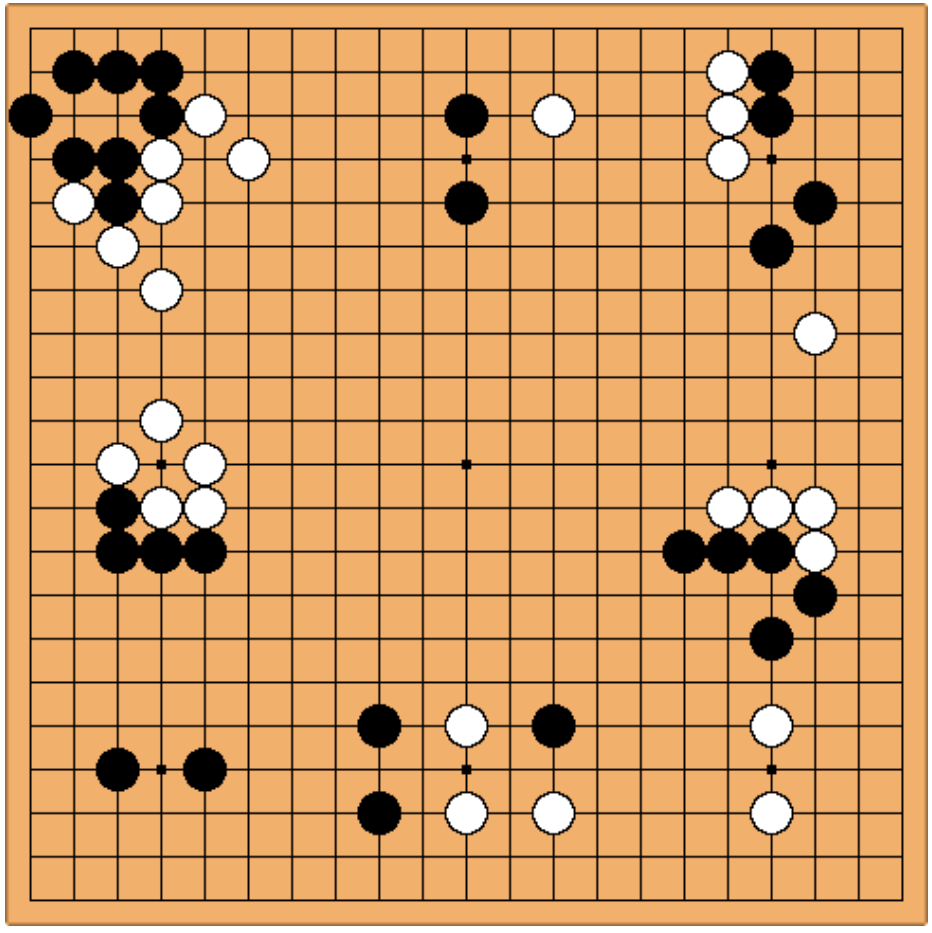
- Next two lectures touch on ML for data having graphical structure
- This is quite common



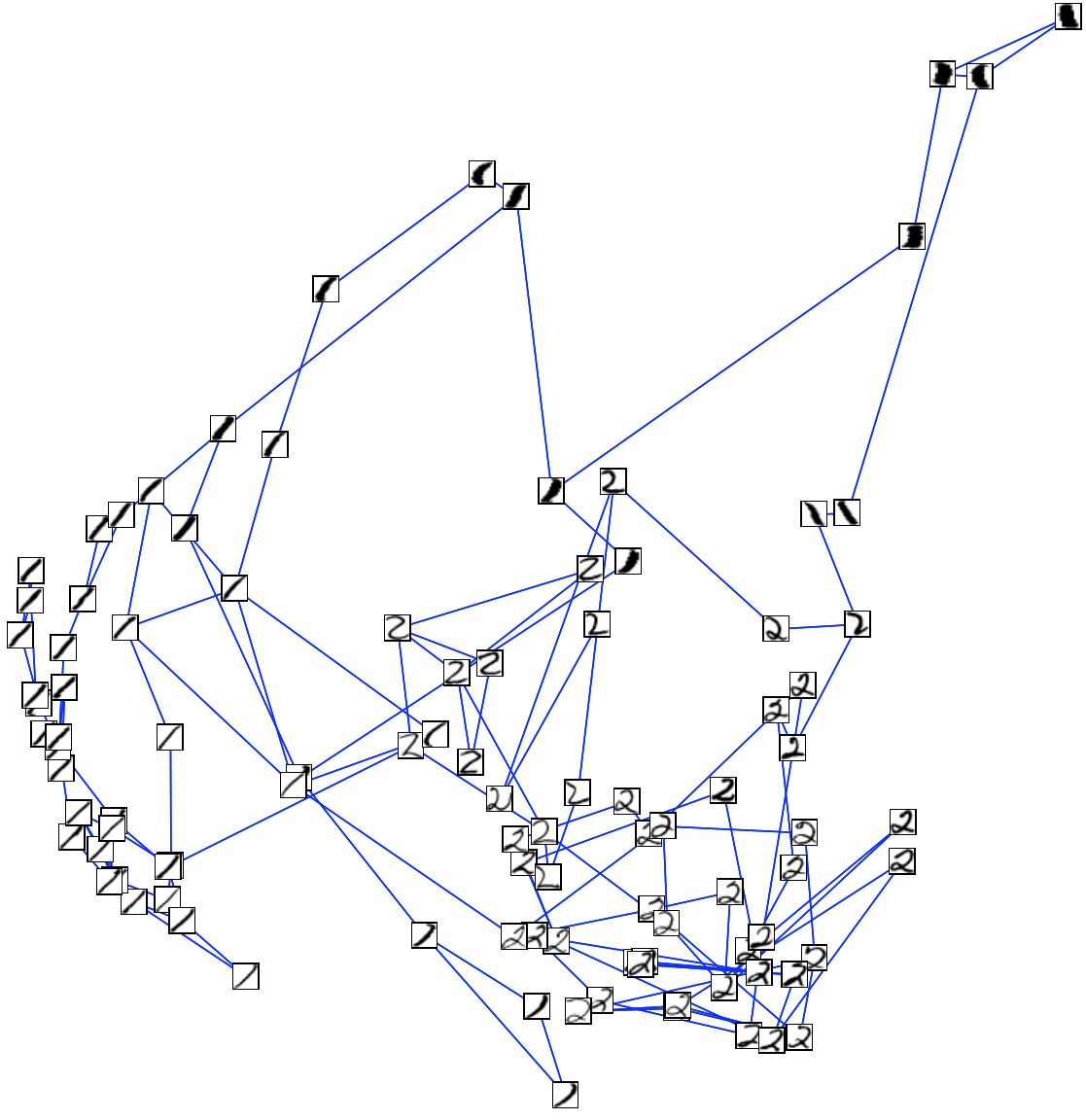
social networks (Facebook friends graph)



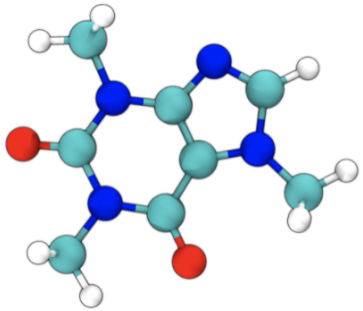
gene interaction networks — https://en.wikipedia.org/wiki/Interactome

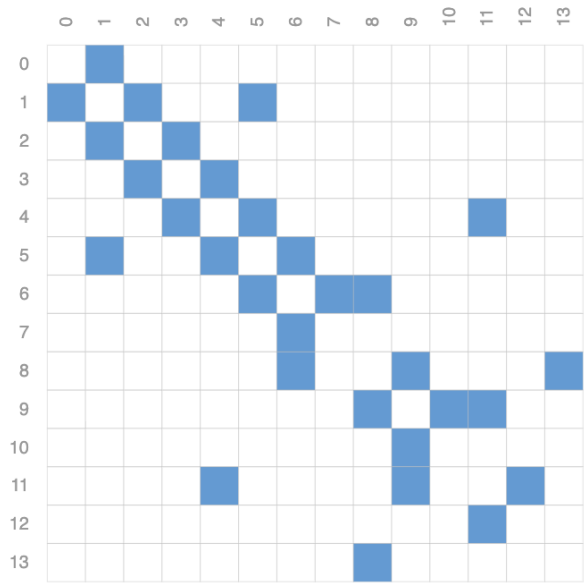


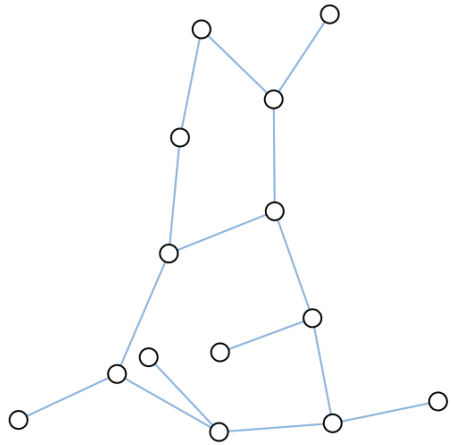
games



semi-supervised learning

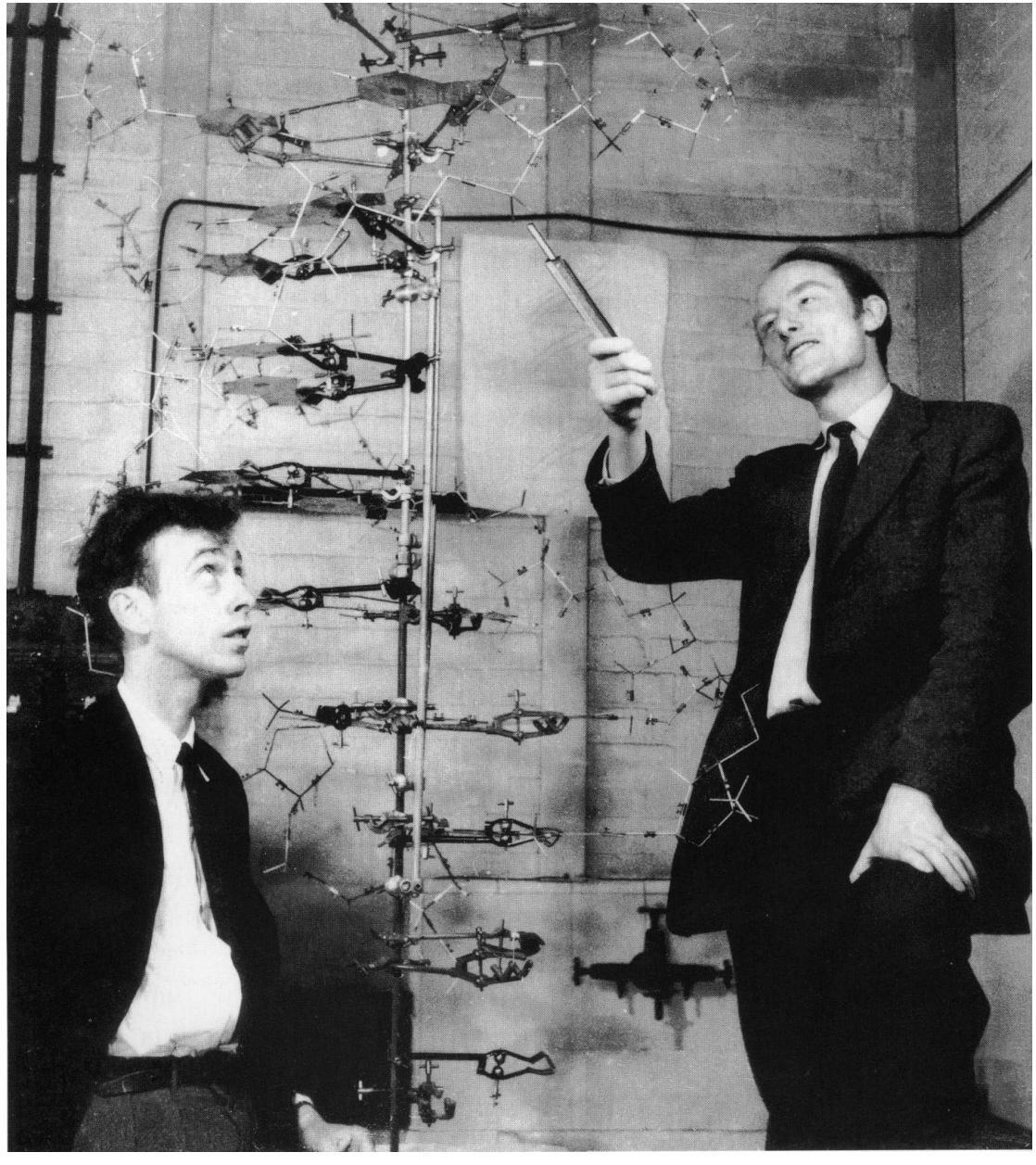


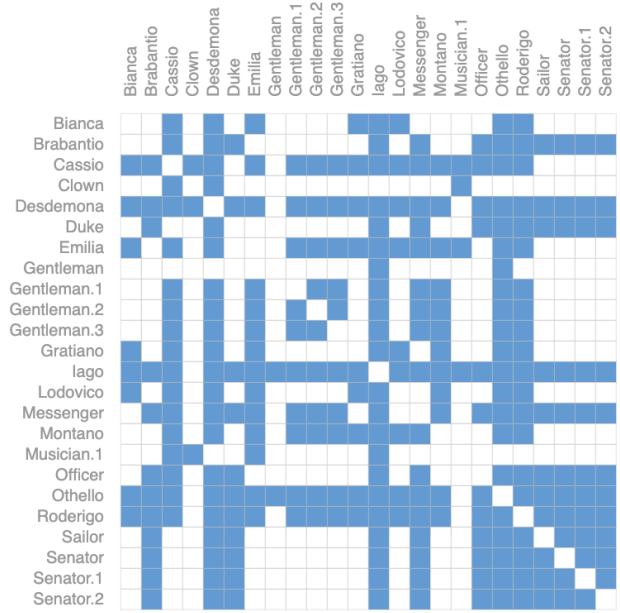


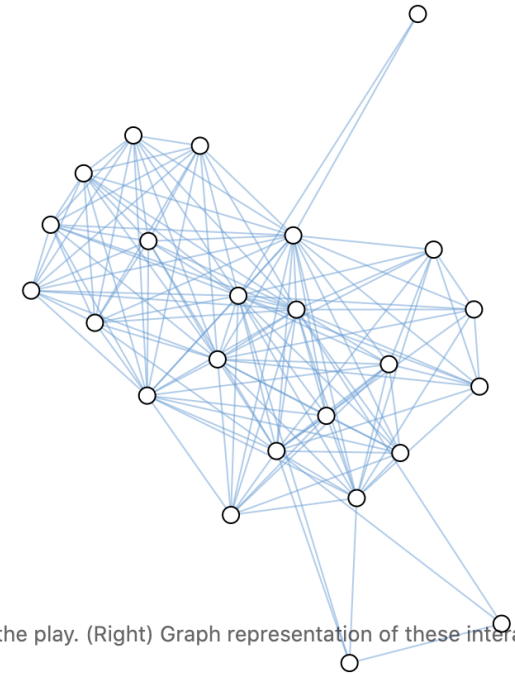


(Left) 3d representation of the Caffeine molecule (Center) Adjacency matrix of the bonds in the molecule (Right) Graph representation of the molecule.

https://distill.pub/2021/gnn-intro/



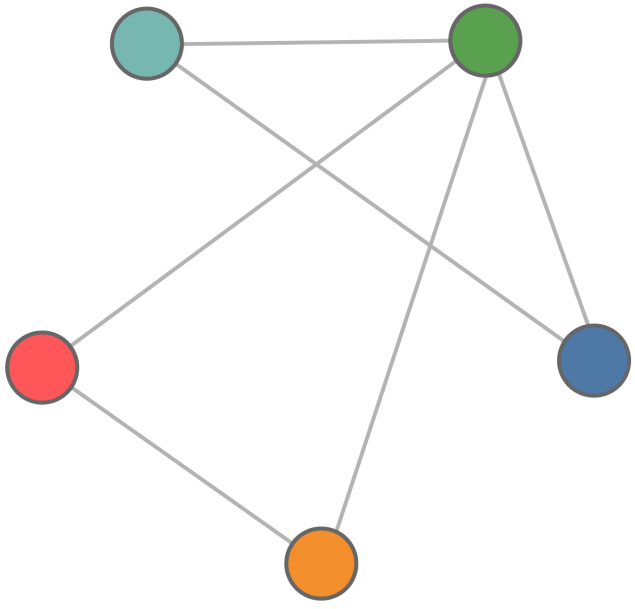




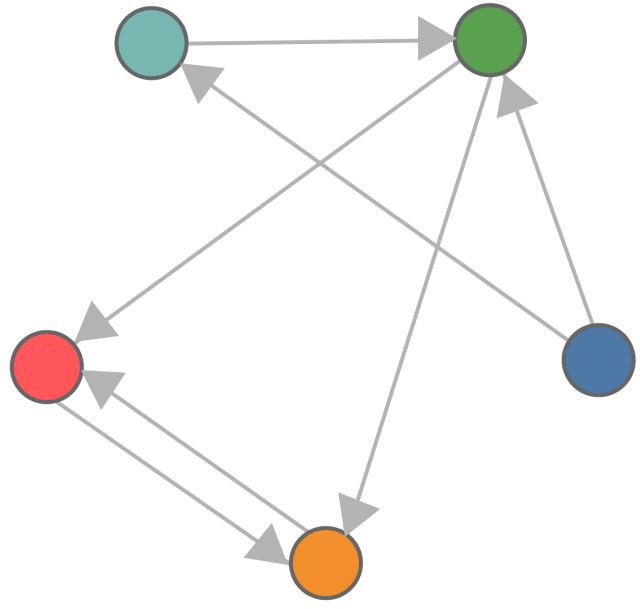
(Left) Image of a scene from the play "Othello". (Center) Adjacency matrix of the interaction between characters in the play. (Right) Graph representation of these interactions.

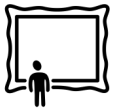
https://distill.pub/2021/gnn-intro/

Undirected graph



Directed graph





Graphs

- A natural language for describing various data
- Give information about relationships between variables
- Associated with each multivariate distribution

Undirected Graphs

A graph $G = (V, E)$ has vertices V, edges E.

If $X = (X_1, \ldots, X_p)$ is a random variable, we will study graphs where there are p vertices, one for each X_j.

The graph will encode conditional independence relations among the variables.

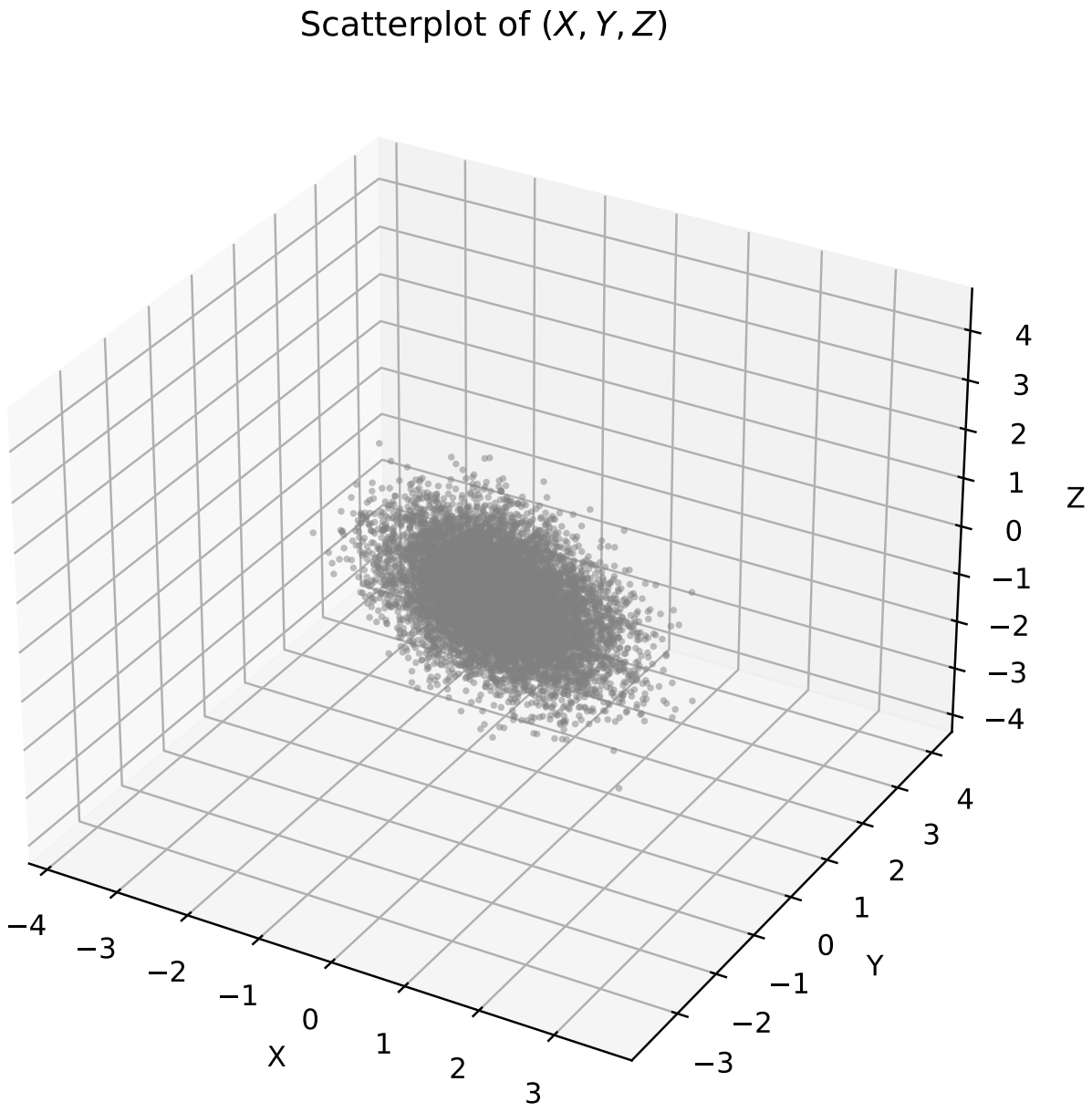
Example: Gaussian data

We have a three-dimensional Gaussian (X, Y, Z) with covariance

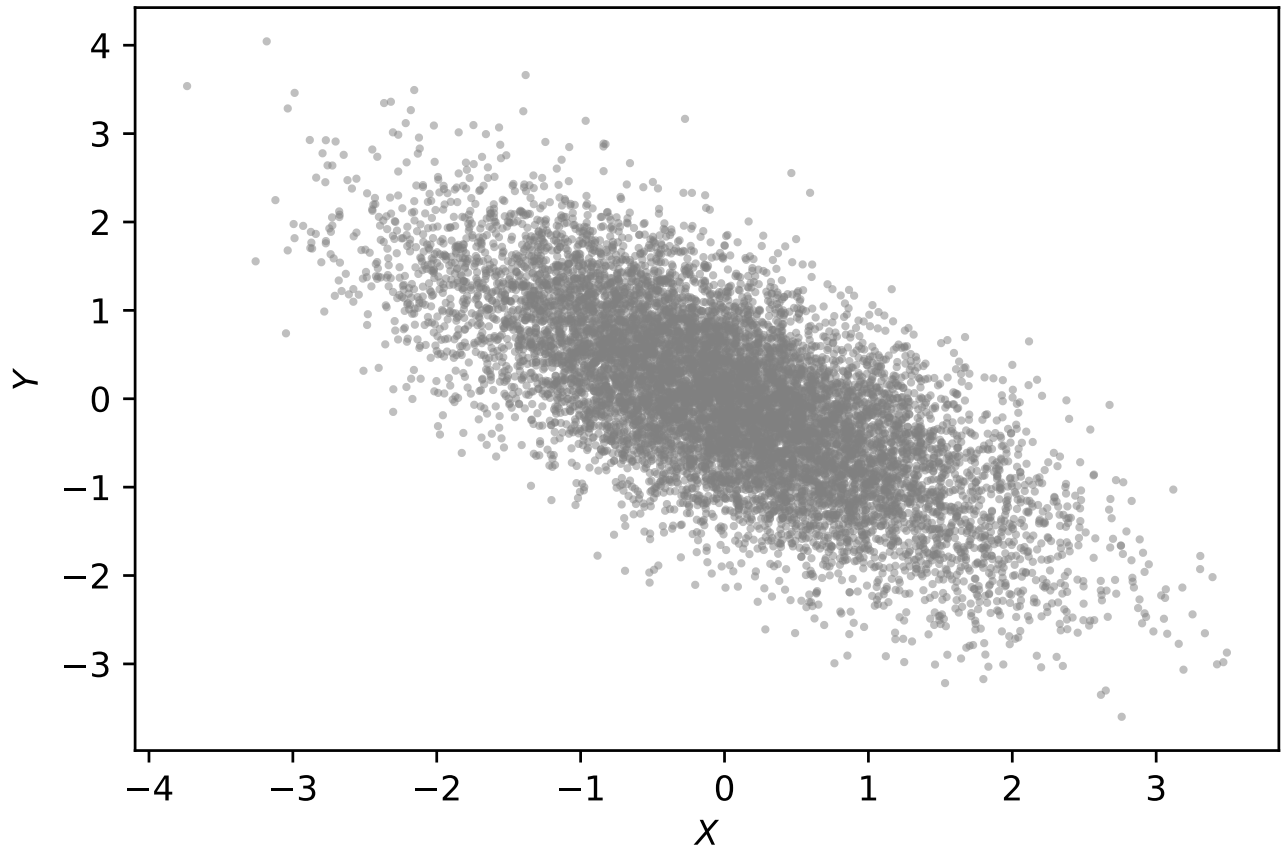
$$\Sigma = \begin{pmatrix} 3.1 & -2.4 & 2.1 \\ -2.4 & 2.6 & -2.4 \\ 2.1 & -2.4 & 3.1 \end{pmatrix}$$

So, all pairs are correlated

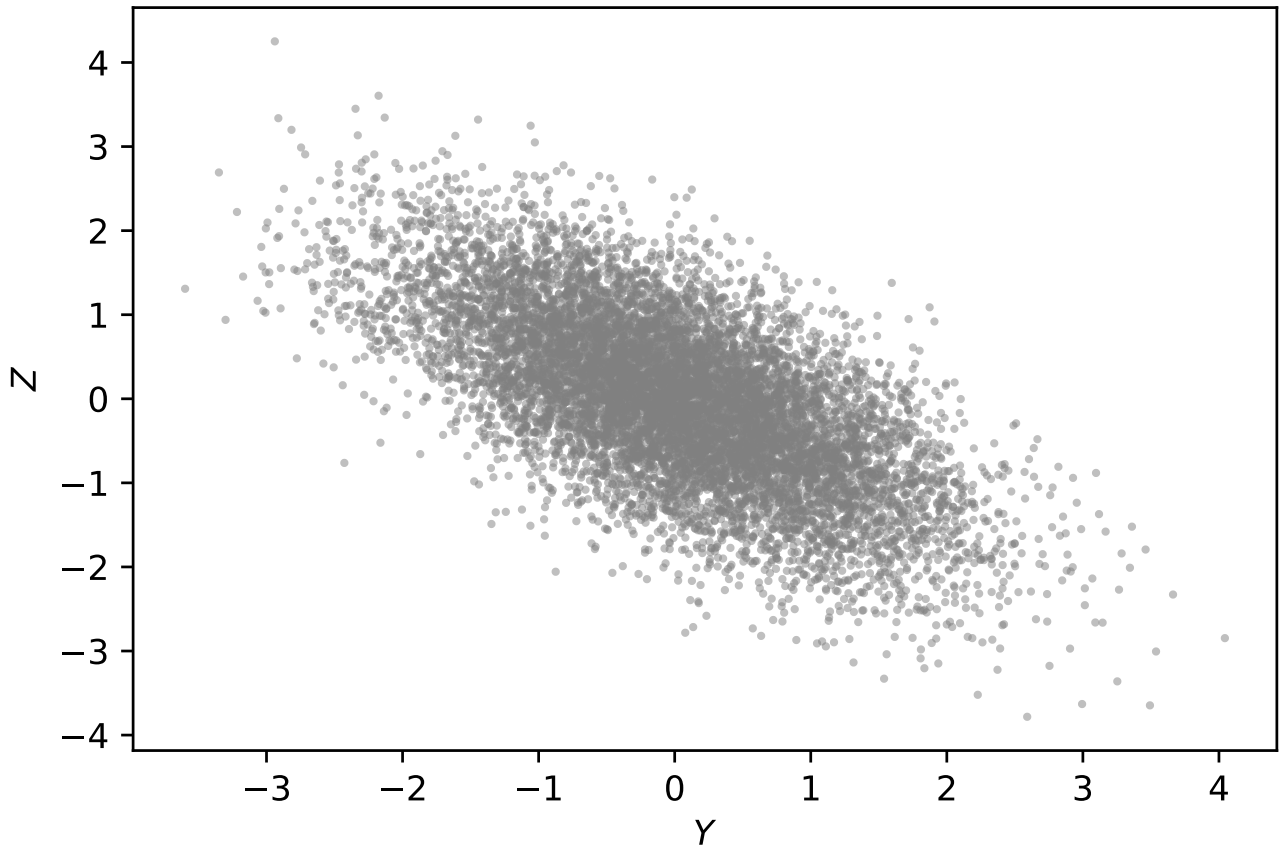
Example: Gaussian data



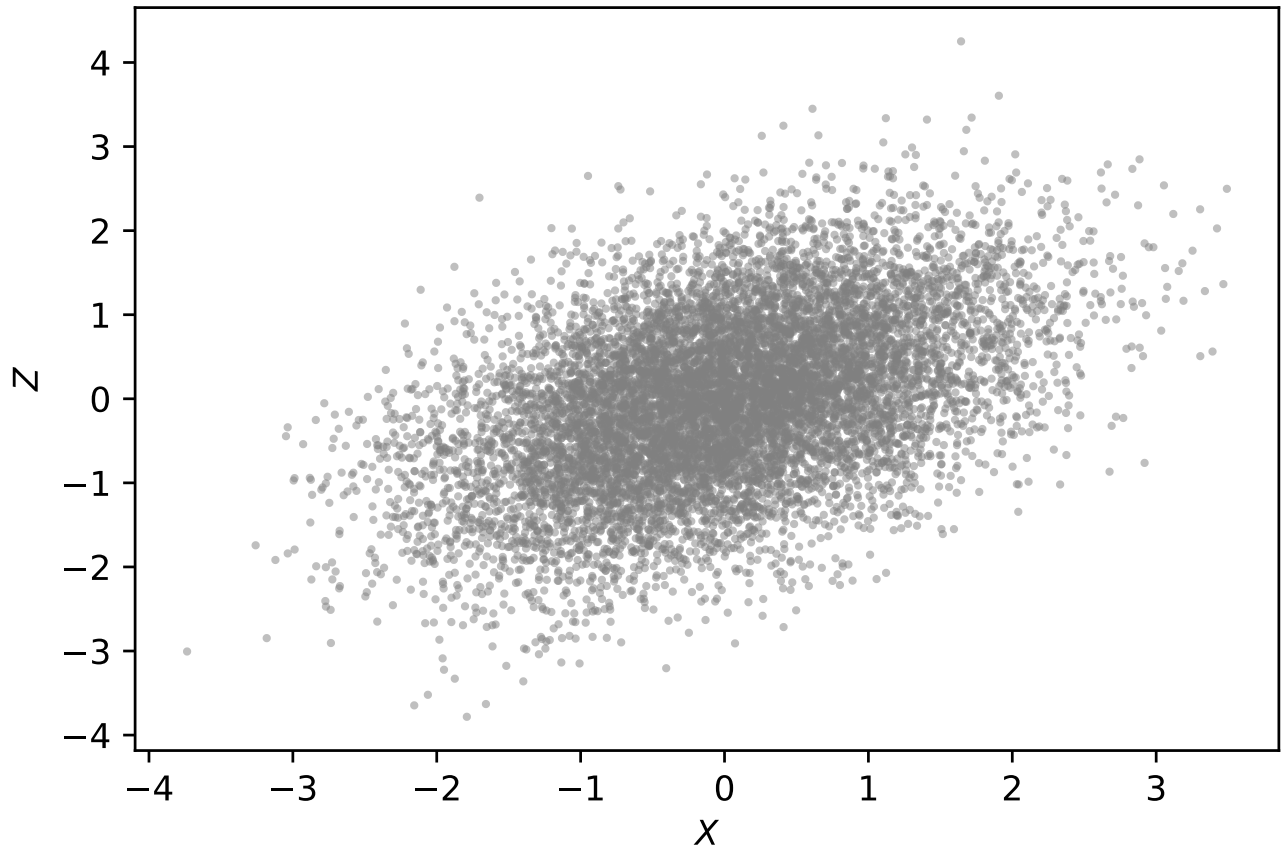
Example: Gaussian data



Example: Gaussian data

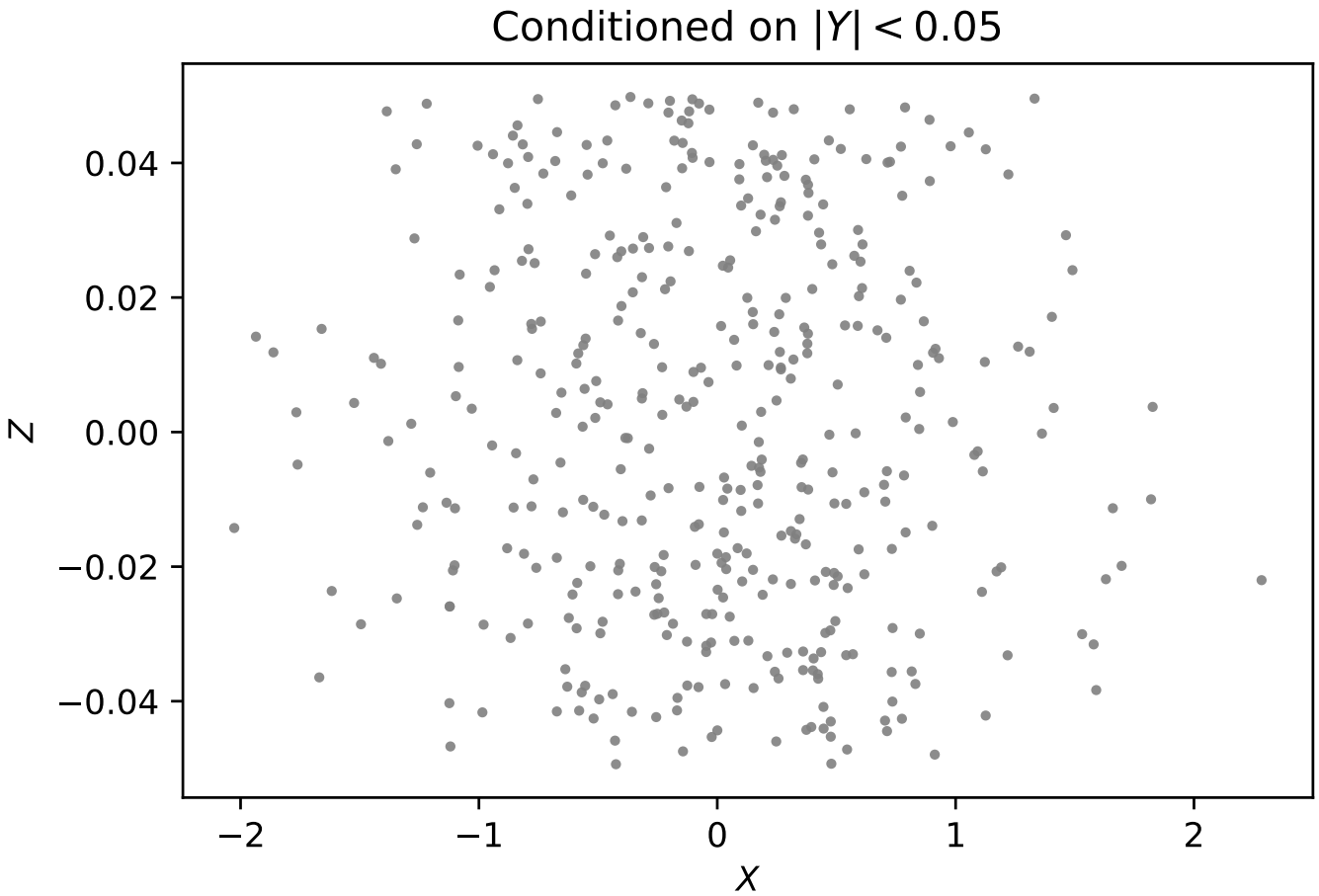


Example: Gaussian data



Example: Gaussian data

But when we condition on $Y \approx 0$:



Gaussian example

This is revealed in the “precision matrix”

$$\Omega \equiv \Sigma^{-1} = \begin{pmatrix} 1 & \frac{9}{10} & 0 \\ \frac{9}{10} & 2 & \frac{9}{10} \\ 0 & \frac{9}{10} & 1 \end{pmatrix}$$

The zeros indicate conditional independence assumptions

Undirected graphs

Simplest case:

Here $V = \{X, Y, Z\}$ and $E = \{(X, Y), (Y, Z)\}$.

This encodes the independence relation

$$X \perp\!\!\!\perp Z \mid Y$$

which means that *X and Z are independent conditioned on Y*.

Markov Property

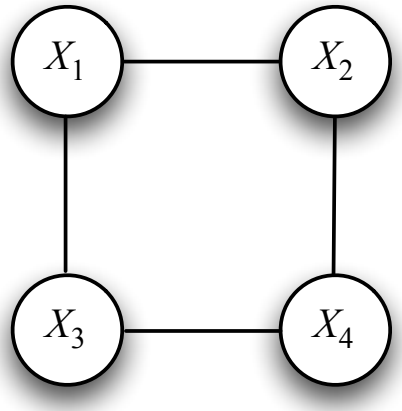
A probability distribution P satisfies the *global Markov property* with respect to a graph G if:

for any disjoint vertex subsets A, B, and C such that C separates A and B,

$$X_A \perp\!\!\!\perp X_B \mid X_C.$$

- X_A are the random variables X_j with $j \in A$.
- C separates A and B means that there is no path from A to B that does not pass through C.

Example

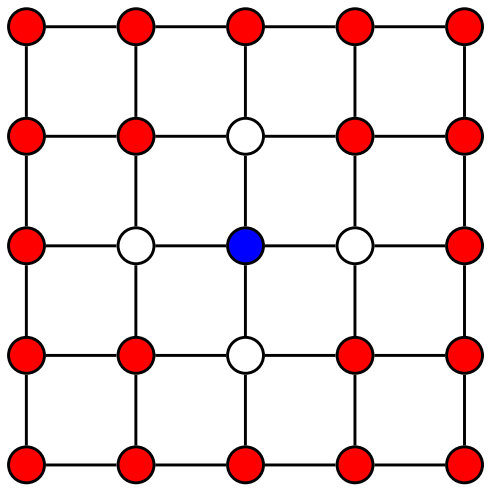


$$X_1 \perp\!\!\!\perp X_4 \mid X_2, X_3$$

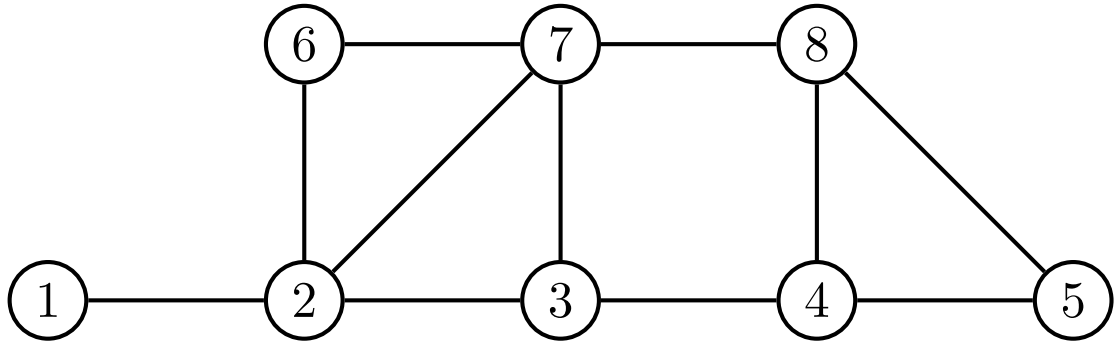
$$X_2 \perp\!\!\!\perp X_3 \mid X_1, X_4$$

Example: 2-dimensional grid

The blue node is independent of the red nodes given the white nodes.



Example



$C = \{3, 7\}$ separates $A = \{1, 2\}$ and $B = \{4, 8\}$. Hence,

$$\{X_1, X_2\} \perp\!\!\!\perp \{X_4, X_8\} \quad \Big| \quad \{X_3, X_7\}$$

Special case

If $(i, j) \notin E$ then

$$X_i \perp\!\!\!\perp X_j \mid \{X_k : k \neq i, j\}$$

You'll prove this on Assn 3.

Special case

If $(i, j) \notin E$ then

$$X_i \perp\!\!\!\perp X_j \mid \{X_k : k \neq i, j\}$$

Lack of an edge from i to j implies that X_i and X_j are independent given all of the other random variables.

You'll prove this on Assn 3.

Graph estimation

- A graph G represents the class of distributions, $\mathcal{P}(G)$, the distributions that are Markov with respect to G
- Graph estimation: Given n samples $X_1, \ldots, X_n \sim P$, estimate the graph G.

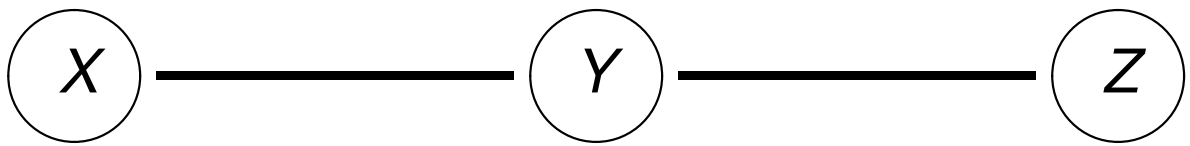
Gaussian case

Let $\Omega = \Sigma^{-1}$ be the precision matrix.

A zero in Ω indicates a *lack of the corresponding edge* in the graph

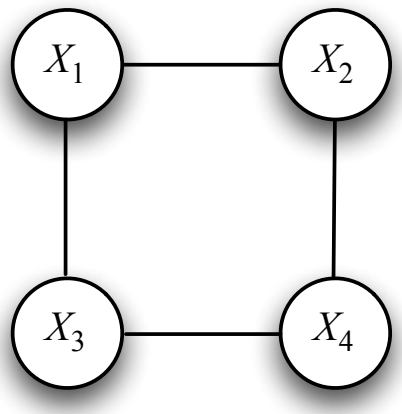
Gaussian case

$$\Omega \equiv \Sigma^{-1} = \begin{pmatrix} * & * & 0 \\ * & * & * \\ 0 & * & * \end{pmatrix}$$



Gaussian case

$$\Omega \equiv \Sigma^{-1} = \begin{pmatrix} * & * & * & 0 \\ * & * & 0 & * \\ * & 0 & * & * \\ 0 & * & * & * \end{pmatrix}$$



$$X_1 \perp\!\!\!\perp X_4 \mid X_2, X_3$$

The machine learning problem

How do we estimate the graph from a sample of data?

Gaussian case: Algorithms

Two approaches:

- parallel lasso
- graphical lasso

Parallel Lasso:

1. For each $j = 1, \ldots, p$ (in parallel): Regress X_j on all other variables using the lasso.
2. Put an edge between X_i and X_j if each appears in the regression of the other.

Graphical Lasso (glasso)

- Assume a multivariate Gaussian model
- Subtract out the sample mean
- Minimize the negative log-likelihood of the data, subject to a constraint on the sum of the absolute values of the inverse covariance

Graphical Lasso (glasso)

The glasso optimizes the parameters of $\Omega = \Sigma^{-1}$ by minimizing:

$$\text{trace}(\Omega S_n) - \log|\Omega| + \lambda \sum_{j \neq k} |\Omega_{jk}|$$

where $|\Omega|$ is the determinant and S_n is the sample covariance

$$S_n = \frac{1}{n} \sum_{i=1}^{n} x_i x_i^T$$

Derivation: Where does this come from?

Assume mean is zero. Then the probability density at a data point x is

$$\begin{aligned} p(x) &= \frac{1}{\sqrt{(2\pi)^p |\Sigma|}} \exp\left(-\frac{1}{2} x^T \Sigma^{-1} x\right) \\ &= \frac{1}{\sqrt{(2\pi)^p |\Sigma|}} \exp\left(-\frac{1}{2} x^T \Omega x\right) \\ &= \frac{1}{\sqrt{(2\pi)^p |\Sigma|}} \exp\left(-\frac{1}{2} \text{trace}(\Omega x x^T)\right) \end{aligned}$$

Therefore, using $\log |A| = -\log |A^{-1}|$, up to an additive constant,

$$-\log p(x) = \frac{1}{2} \log |\Sigma| + \frac{1}{2} \text{trace}(\Omega x x^T) = -\frac{1}{2} \log |\Omega| + \frac{1}{2} \text{trace}(\Omega x x^T)$$

Derivation: Where does this come from?

Summing over all the data we have

$$-\sum_{i=1}^{n} \log p(x_i) = \frac{1}{2}\sum_{i=1}^{n} \text{trace}(\Omega x_i x_i^T) - \frac{n}{2}\log|\Omega|$$
$$= \frac{n}{2}\text{trace}(\Omega S_n) - \frac{n}{2}\log|\Omega|$$

Rescaling by $2/n$ and adding the ℓ_1 penalty, we get the objective function

$$\mathcal{O}(\Omega) = \text{trace}(\Omega S_n) - \log|\Omega| + \lambda \sum_{k \neq j} |\Omega_{jk}|$$

This is a convex function of Ω

Graphical Lasso (glasso)

There is a simple blockwise gradient descent algorithm for minimizing this function. It is similar to the algorithm for the lasso that we studied.

Python packages: `sklearn.covariance.GraphicalLasso` and `sklearn.covariance.GraphicalLassoCV`

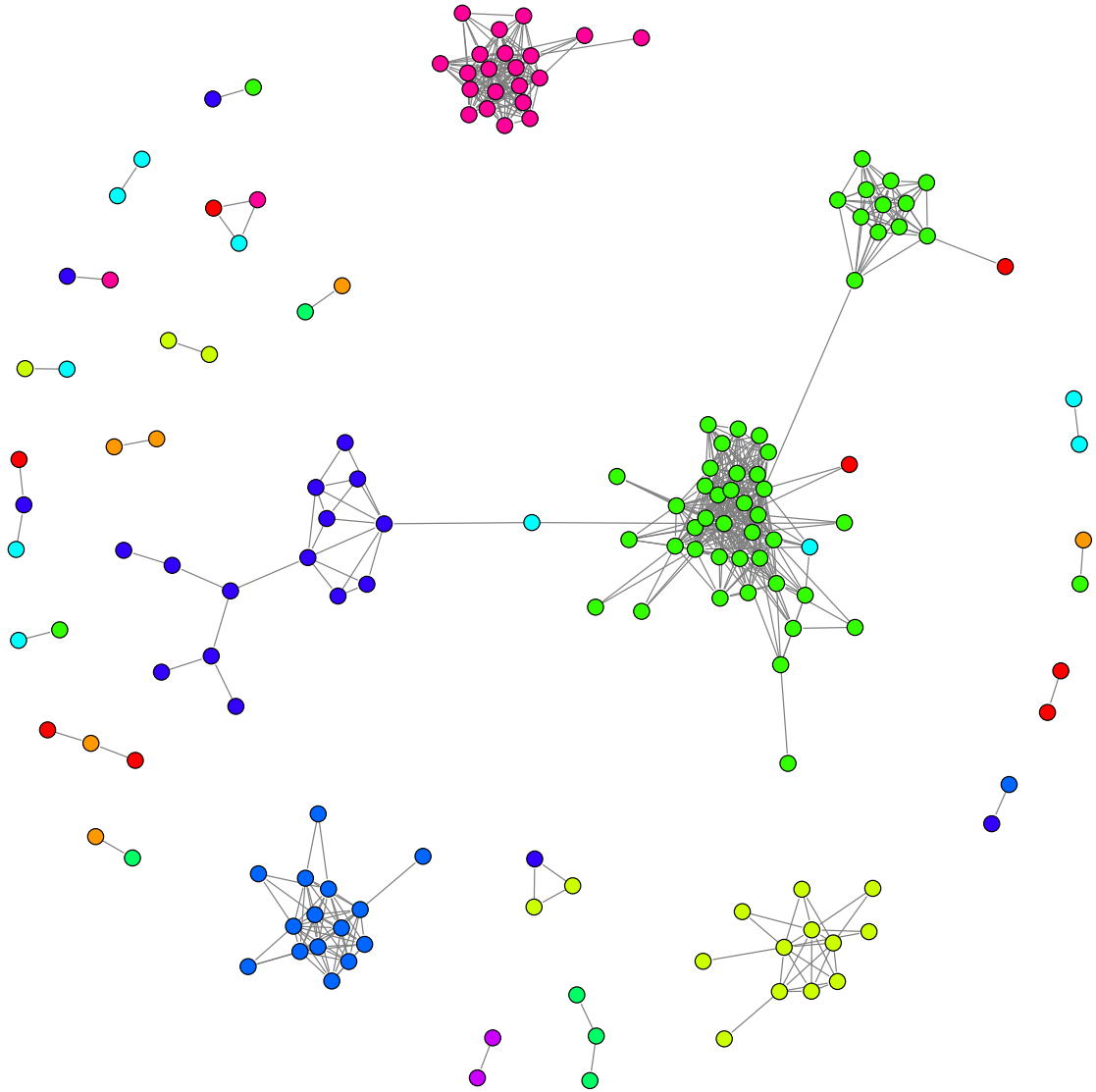
For a derivation of this algorithm, see “Notes on Graphs and Structure Learning”

Graphs on the S&P 500

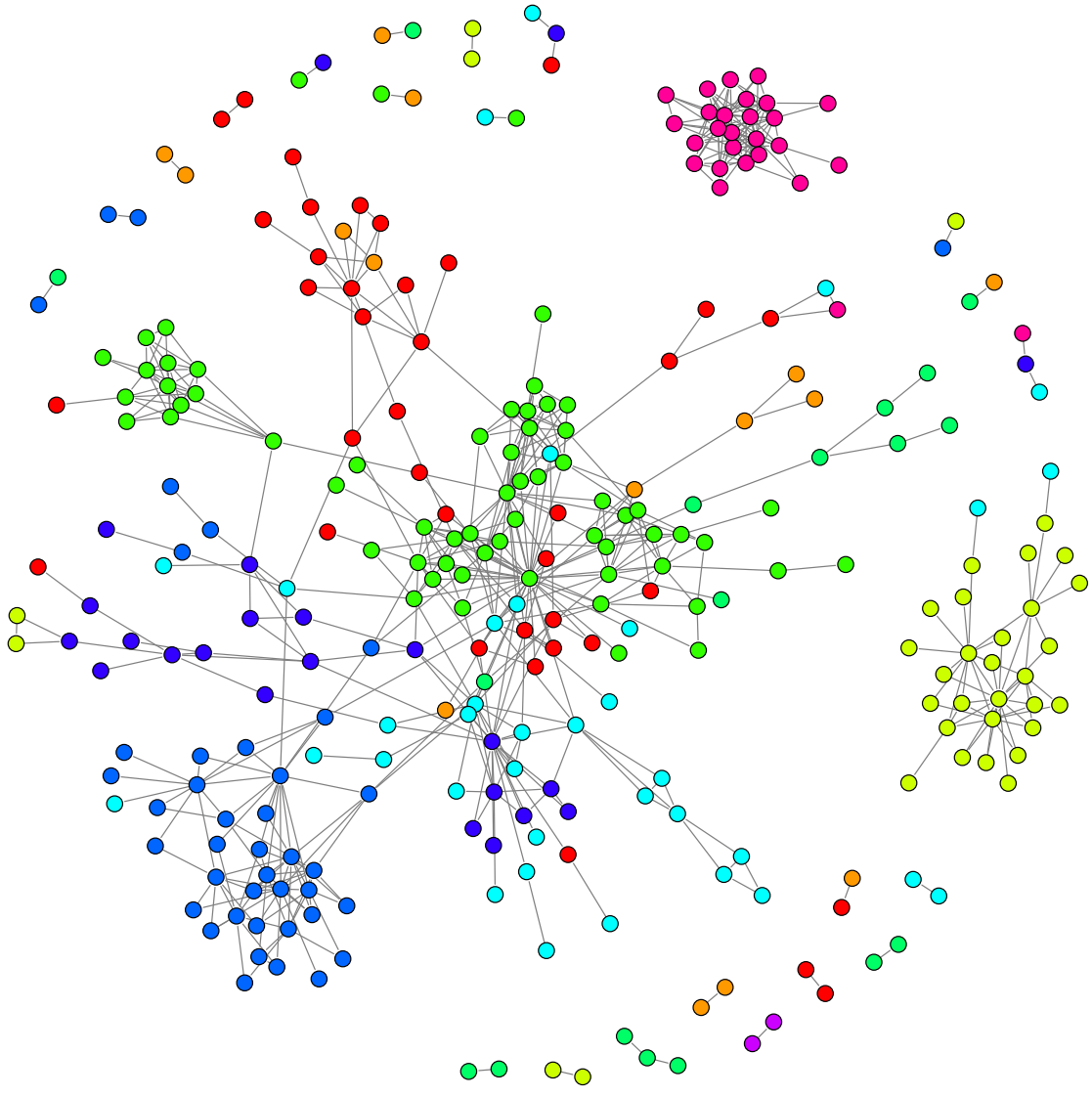
- Data from Yahoo! Finance (`finance.yahoo.com`).
- Daily closing prices for 452 stocks in the S&P 500 between 2003 and 2008 (before onset of the "financial crisis").
- Log returns $X_{tj} = \log\left(S_{t,j}/S_{t-1,j}\right)$.
- Outliers capped at $\pm 6\sigma$.
- In following graphs, each node is a stock, and color indicates an industry sector

```
Consumer Discretionary          Consumer Staples
Energy                          Financials
Health Care                     Industrials
Information Technology          Materials
Telecommunications Services     Utilities
```

S&P 500: Graphical Lasso



S&P 500: Parallel Lasso



Example Neighborhood

Yahoo Inc. (Information Technology):

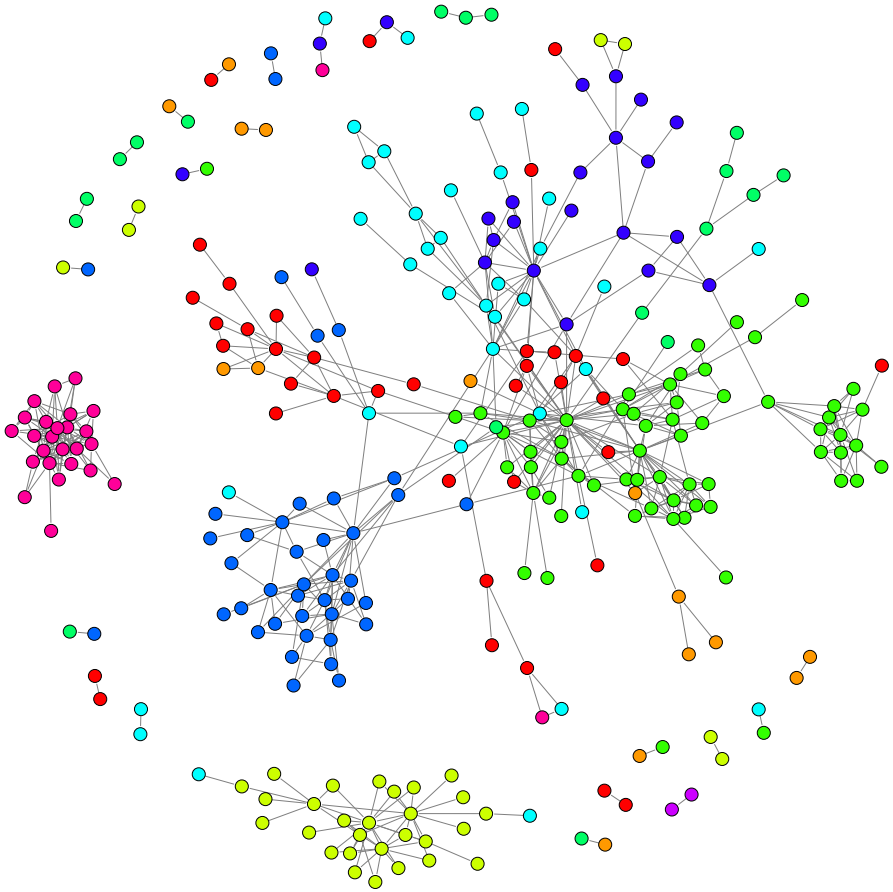
- Amazon.com Inc. (Consumer Discretionary)
- eBay Inc. (Information Technology)
- NetApp (Information Technology)

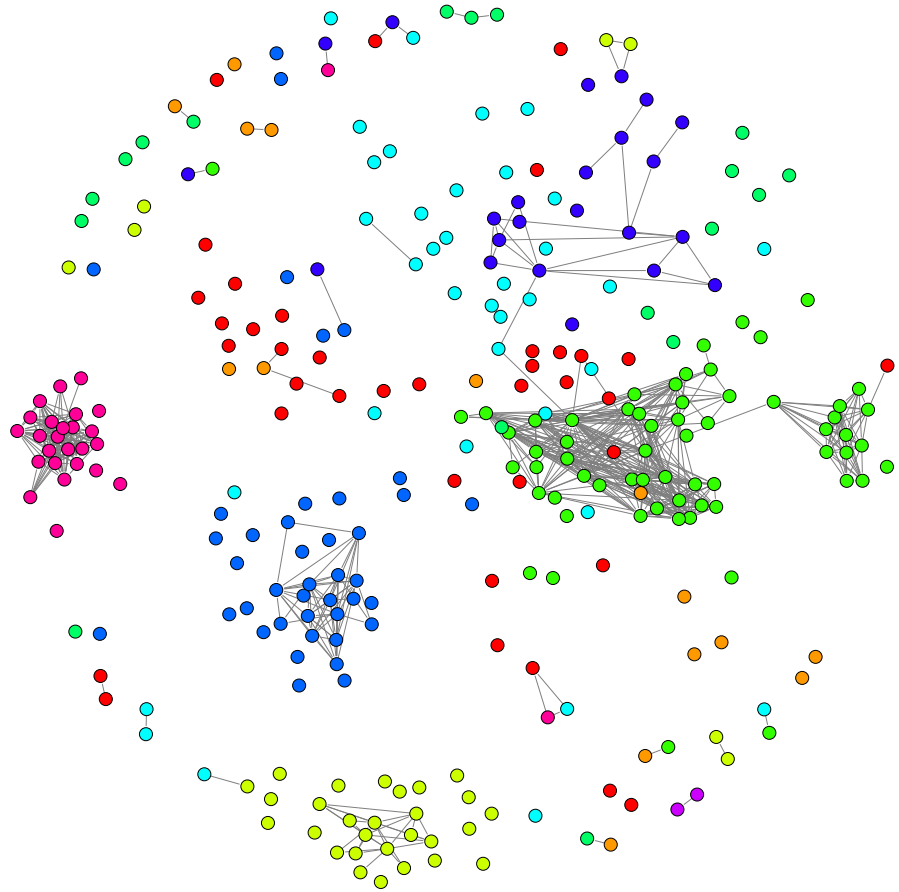
Example Neighborhood

Target Corp. (Consumer Discretionary):

- Big Lots, Inc. (Consumer Discretionary)
- Costco Co. (Consumer Staples)
- Family Dollar Stores (Consumer Discretionary)
- Kohl's Corp. (Consumer Discretionary)
- Lowe's Cos. (Consumer Discretionary)
- Macy's Inc. (Consumer Discretionary)
- Wal-Mart Stores (Consumer Staples)

Parallel vs. Graphical





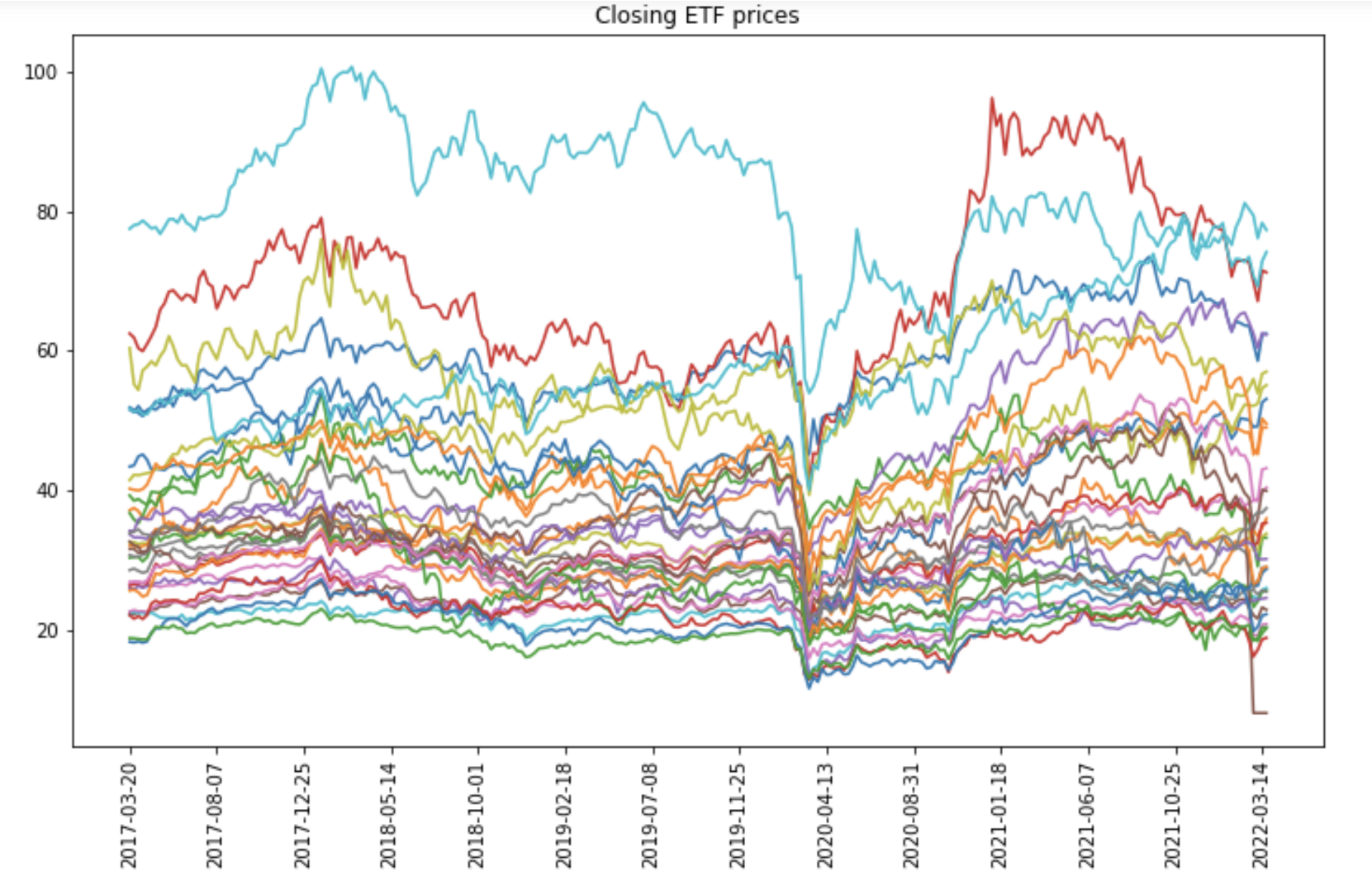
Choosing λ

Can use:

1. Cross-validation
2. BIC = log-likelihood - $(p/2)\log n$
3. AIC = log-likelihood - p

where p = number of parameters.

Let’s go to the demo!



Summary

- Graphs encode conditional independence assumptions
- Sparse graphs represent low-dimensional structure in high dimensional data
- Gaussian case: Graph read off from precision matrix
- Graphical lasso used to estimate the graph